ENG 6010: AI in Robotics

Homework 4: Deep Learning, Convolutional Neural Networks (CNNs) & Architectures

Instructor: Dr. Hongbo Zhang

Total Points: 100 Points

Student Information

Student Name: _____

Instructions & Submission Guidelines

- Show all mathematical calculations, receptive field proofs, and parameter counts step-by-step for Sections I and II.
- Round floating-point calculation results to 4 decimal places where applicable.
- Submit your completed solution document (`.pdf`) along with your Python source code scripts for Section III.

Part I: Conceptual & Mathematical Fundamentals (30 Points)

Problem 1.1: Convolutional Mechanics, Activation Functions & Residual Gradient Flow (30 Points)

Define the general formulas for spatial output dimensions $(W_{out}, H_{out}, D_{out})$ and total learnable parameters in a 2D convolutional layer processing an input volume $W_{in} \times H_{in} \times D_{in}$ with K filters of size $F \times F$, stride S, and padding P, explaining how parameter sharing contrasts with fully connected layers; then compare Sigmoid, Tanh, and ReLU activation functions regarding saturation and the dying ReLU problem with Leaky ReLU's fix; prove that stacking two 3×3 conv layers achieves an identical effective receptive field to one 5×5 layer with 28% fewer parameters; and derive the gradient $\frac{\partial \mathcal{E}}{\partial x}$ through a ResNet residual block $y = \mathcal{F}(x, \{W_i\}) + x$ to mathematically demonstrate how identity shortcut connections prevent vanishing gradients during backpropagation.

Part II: Numerical Calculations (40 Points)

Problem 2.1: 2D Convolution, Max Pooling & Layer Architecture Accounting (40 Points)

Given a 4×4 single-channel input matrix I and a 3×3 horizontal edge kernel K, compute the valid 2D convolution $O = I * K$ ($S = 1, P = 0$); perform 2×2 Max Pooling ($S = 2$) on a given 4×4 activation matrix A; then evaluate a 5-layer CNN architecture processing an RGB image of size $224 \times 224 \times 3$ consisting sequentially of Conv1 ($K = 64, F = 7, S = 2, P = 3$), Pool1 ($F = 3, S = 2$), Conv2 ($K = 128, F = 3, S = 1, P = 1$), Pool2 ($F = 2, S = 2$), and FC1 (512 output neurons), calculating the output spatial volume shape ($W_{out} \times H_{out} \times D_{out}$) and learnable parameter count for each layer, along with the total update iterations in one epoch for $N = 60,000$ images at mini-batch size $B = 128$.

$$I = \begin{bmatrix} 1 & 2 & 0 & 2 \\ 3 & 1 & 4 & 1 \\ 0 & 2 & 3 & 0 \\ 1 & 0 & 1 & 2 \end{bmatrix}, \qquad K = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}, \qquad A = \begin{bmatrix} 12 & 8 & 3 & 15 \\ 4 & 19 & 7 & 2 \\ 0 & 14 & 22 & 9 \\ 6 & 11 & 5 & 18 \end{bmatrix}$$

Part III: Programming & Implementation (30 Points)

Problem 3.1: Custom PyTorch CNN Architecture & Transfer Learning (30 Points)

Implement a Python script using PyTorch that defines a custom 3-layer Convolutional Neural Network (`nn.Conv2d`, `nn.MaxPool2d`, `nn.Linear`) trained on CIFAR-10/MNIST with CrossEntropyLoss and the Adam optimizer, and implement a transfer learning script using pre-trained ResNet-18 (`torchvision.models.resnet18(pretrained=True)`) where feature extraction backbone weights are frozen (`requires_grad = False`) and the final classification head is replaced to fit a target dataset.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class CustomCNN(nn.Module):
    def __init__(self, num_classes=10):
        super(CustomCNN, self).__init__()
        self.conv1 = nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)
        self.conv2 = nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(64 * 8 * 8, 128)
        self.fc2 = nn.Linear(128, num_classes)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

Listing 1: PyTorch Starter Code Template